

CSC331: Data Structures - BMCC Fall 2025
Professor Byron
Program #5 specifications: Minimum spanning tree
Date Due: 1159pm Fri December 12, 2025; no credit for late submission

Your task for this assignment is to identify a minimum spanning tree (MST) in a connected undirected weighted graph using C++.

1. Implement Prim's minimum spanning tree (MST) algorithm using C++.
2. Use the completed graph_f25.h header file (modified from the Malik textbook page 706), incomplete printGraph.h header file and incomplete genFromFile.h header file. These header files are available in Brightspace. The header files are to be kept separate from your main C++ program and accessed using "include" statements. No changes are needed to the graph_f25.h header file.
3. Code a C++ program to prompt the user for a number of vertices. Use the graphType class constructor to create a new graph instance with the number of vertices entered by the user. Run the genRandom method to assign random edges and weights for the new graph instance. Run the minimumSpanning method to identify a minimum spanning tree in the new graph instance. Run the printTreeAndWeight method to display the minimum spanning tree on the console. Run the ~graphType destructor to deallocate the graph instance.
4. Complete the printGraph.h header file for a void method called printGraph to display the values in the graph on the console in a row/column format. Run the printGraph method after running the GenRandom method and before running the minimumSpanning method.
5. Allow the program to obtain a filename from the command line. The file with the given filename will contain edge transactions. Each input transaction represents an undirected edge of a connected weighted graph. The edge consists of two unequal integers (1 or 2 digits each), representing graph vertices that the edge connects, followed by a non-negative integer edge weight. Graph vertices will start with zero and will be consecutively numbered. The three values on each input transaction are separated by a single space. An input transaction containing the string "end-of-file" signals the end of the graph edge input. The input data is assumed to be valid. There is no need to perform data validation on the input data. Create an integer function named findGraphSize in your main program which has a filename as an argument and returns an integer value representing the number of vertices in the graph described by the edges in the file. If your program detects a command line argument, it will display the filename, run the findGraphSize function and display the number of vertices in the graph described by the edges in the file.
6. Complete the genFromFile.h header file for a void method called genFromFile which has a filename as an argument. The genFromFile method will assign values to the graph from the

file having the filename provided. In the case where a filename is provided at the command line, your program will (1) run the findGraphSize method to determine the number of vertices in the graph in the file, (2) create a new graph instance with that number of vertices, (3) run the genFromFile method to assign values to the new graph instance, (4) run the printGraph method to display the values in the graph, (5) run the minimumSpanning method to find a minimum spanning tree, (6) run the printTreeAndWeight method to display the minimum spanning tree found and (7) run the ~graphType destructor to deallocate the graph instance.

7. Sample run after completing paragraph 3:

```
./prog5
enter number of vertices ... 6
Source Vertex: 0
Edges Weight
(2, 1) 4
(4, 2) 2
(0, 3) 9
(0, 4) 1
(1, 5) 12

Minimum spanning Tree Weight: 28

Graph deleted.
```

8. Sample run after completing paragraph 4:

```
./prog5
enter number of vertices ... 5

Graph:
999  1    6    2    999
1    999  999  3    4
6    999  999  5    7
2    3    5    999  999
999  4    7    999  999

Source Vertex: 0
Edges Weight
(0, 1) 1
(3, 2) 5
(0, 3) 2
(1, 4) 4
```

Minimum spanning Tree Weight: 12

Graph deleted.

9. Sample run with command line argument after completing paragraph 5:

```
$ ./prog5 graph1.dat
```

Filename: graph1.dat

Number of vertices: 5

10. Sample run with command line argument after completing paragraph 6:

Sample input transactions in a file called graph1.dat are as follows:

```
0 1 1
0 2 6
0 3 2
1 3 3
1 4 4
2 3 5
2 4 7
end-of-file
```

Sample output will be as follows:

```
$ ./prog5 graph1.dat
```

Filename: graph1.dat

Number of vertices: 5

Graph:

999	1	6	2	999
1	999	999	3	4
6	999	999	5	7
2	3	5	999	999
999	4	7	999	999

Source Vertex: 0

Edges Weight

(0, 1) 1

(3, 2) 5

(0, 3) 2

(1, 4) 4

Minimum spanning Tree Weight: 12

Graph deleted.

11. The program will be run at the command prompt by navigating to the directory containing the executable version of the program after the program is compiled.
12. Your C++ program file should be named `csc331_prog5_lastname.cpp`. Your program should contain comments starting on line 1 of the program containing the following information:
 - a. course title, number and section
 - b. your full name
 - c. the program file name
 - d. the program assignment number and due date
 - e. the program purpose
 - f. the programming environment you use (e.g., Visual studio, Repl-it, etc.)

You are encouraged to add additional comments throughout the program that you feel might be helpful to the reader of your source code.

13. Submit your C++ program and any header files you completed (e.g. `genFromFile.h`, `printGraph.h`) as unzipped and unpackaged attachments for an email message to kbyron@bmcc.cuny.edu. Include your name and due date in the completed header files. Use this email subject: "csc331_section_prog5_lastname". A link to a file located on a remote server will not be accepted. If you submit your assignment at least one week early, it will be graded. If you wish, you may then re-submit your complete assignment prior to the due date for a potentially higher score.
14. Do your own work. All students submitting copies of the same program will receive grades of zero for the assignment and may be referred to the Dean of Students, as described in the BMCC plagiarism policy. Do not share your code until you receive your final grade for the course.
15. Grading:
 - a. 0% - your program does not compile.
 - b. 60% - complete paragraph 3 above.
 - c. 80% - complete paragraph 4 above.
 - d. 100% - complete paragraph 5 above.
 - e. 120% - complete paragraph 6 above.